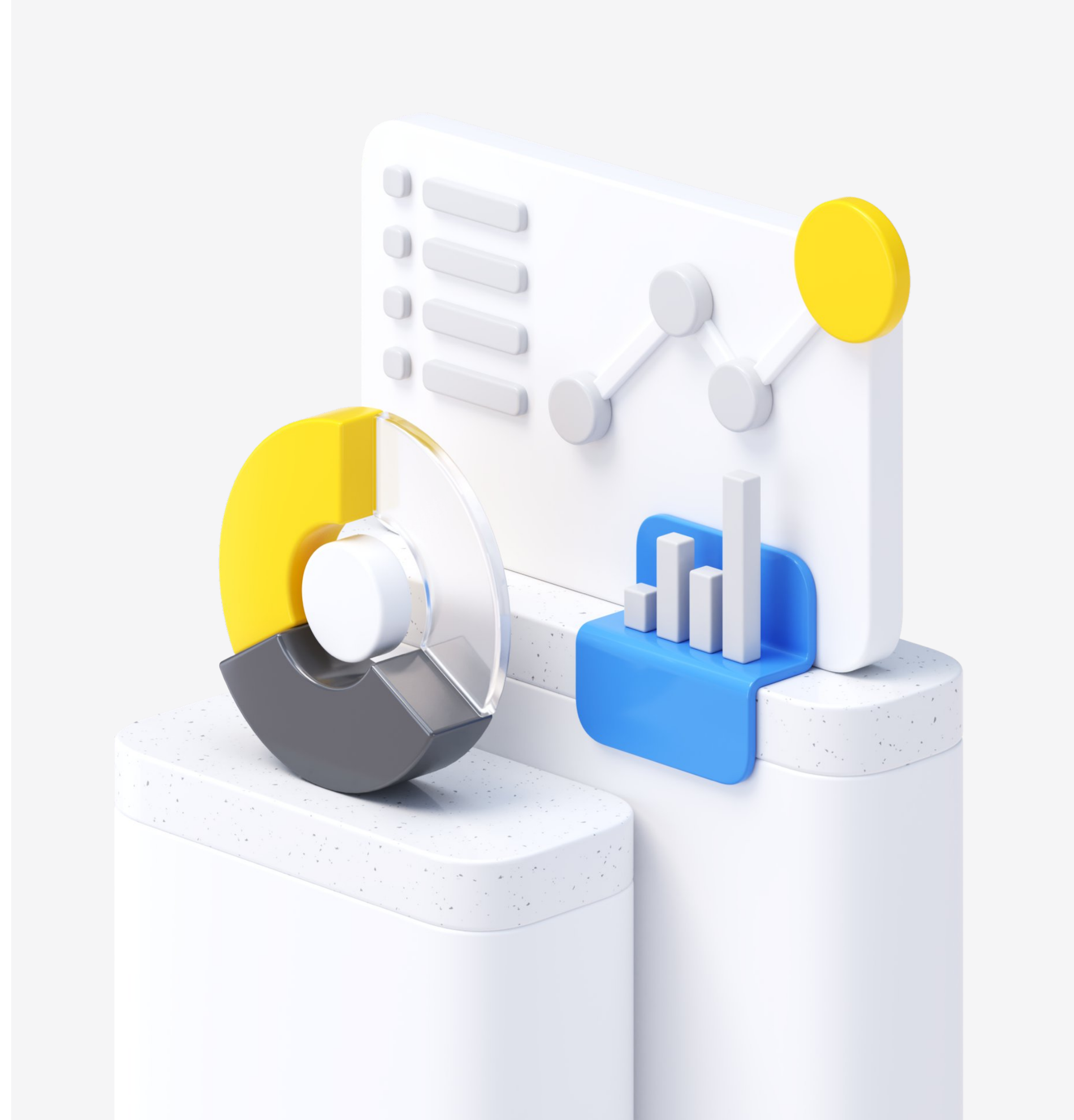


Лекция 4

Swift: наследование, OOP, POP, SOLID



Обо мне



Васькин Михаил

2023

Прошел на летнюю стажировку в Т-Банк

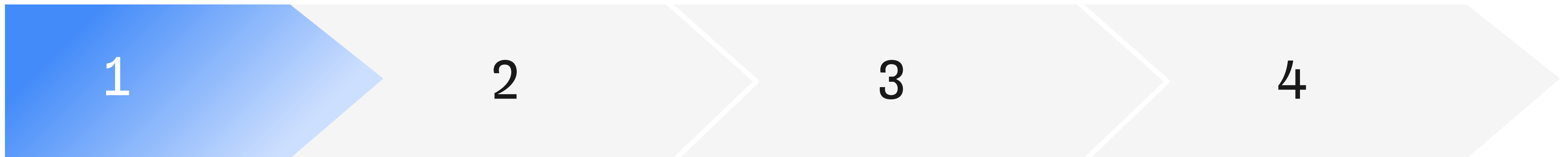
2025

Заканчиваю 4 курс кафедры ИУ5

2025

Работаю iOS-разработчиком в Т-Бизнес

План презентации



OOP и POP

SOLID

Extensions

Any,

AnyObject,

Экзистенциальный
контейнер

Введение в ООП и РОР

Парадигмы программирова ния: от ООП к РОР



ООП

Фокусируется на объектах, которые сочетают в себе данные (свойства) и поведение (методы).



РОР

Ставит в центр внимания протоколы – наборы требований, которым типы могут соответствовать.

Преимущества протокол- ориентированного программирования



Гибкость и Композиция

Протоколы поощряют композицию



Избегание проблем наследования

Код более предсказуемый и легкий в рефакторинге



Меньшая связанность (Decoupling)

Протоколы снижают зависимость между типами.



Лучшая тестируемость

Код, основанный на протоколах, легче тестировать, так как можно создавать "моки" и "стабы"



Подходит для Value Types

POP отлично работает со структурами и перечислениями

Наследование

Наследование: Основы и Концепции



Наследование – это фундаментальный механизм ООП, позволяющий создать новый класс (подкласс или производный класс) на основе существующего класса (суперкласс или базовый класс).

Основные концепции наследования:



Суперкласс (Базовый класс)

Класс, от которого происходит наследование.



Подкласс (Производный класс)

Класс, который наследует свойства и методы суперкласса.



Наследование свойств и методов

Подкласс автоматически получает все не private свойства и методы суперкласса.



Переопределение (Overriding)

Подкласс может предоставлять собственную реализацию методов

Наследование: Базовый пример кода



Наследование – это фундаментальный механизм ООП, позволяющий создать новый класс (подкласс или производный класс) на основе существующего класса (суперкласс или базовый класс).

Lecture_4 > No Selection

```
6 class TransportVehicle {
7     var speed: Double = 0.0
8
9     func move() {
10         print("Transport vehicle is moving
            at speed \(speed) km/h")
11     }
12 }
13
14 class Car: TransportVehicle {
15     var numberOfDoors: Int = 4
16
17     func openDoor() {
18         print("Car door is opened")
19     }
20 }
21
22 class Bicycle: TransportVehicle {
23     var hasBasket: Bool = false
24
25     func ringBell() {
26         print("Ring-ring-ring!")
27     }
28 }
29
30 let transport = TransportVehicle()
31 transport.move()
32
33 let car = Car()
34 car.speed = 60.0
35 car.move()
36 car.openDoor()
37
38 let bicycle = Bicycle()
39 bicycle.speed = 20.0
40 bicycle.move()
41 bicycle.ringBell()
```

☐ "Transport vehicle is moving at speed 20.0 km/h\n"

☐ "Car door is opened\n"

☐ "Ring-ring-ring!\n"

☐ TransportVehicle

☐ TransportVehicle

☐ Car

☐ Car

☐ Car

☐ Car

☐ Bicycle

☐ Bicycle

☐ Bicycle

☐ Bicycle



```
Transport vehicle is moving at speed 0.0 km/h
Transport vehicle is moving at speed 60.0 km/h
Car door is opened
Transport vehicle is moving at speed 20.0 km/h
Ring-ring-ring!
```

Наследование: Переопределение методов

Когда переопределение
полезно:



Специализация поведения

Если подкласс должен вести себя
иначе, чем суперкласс в отношении
определенного метода.



Расширение функциональности

Подкласс может добавить
дополнительную функциональность
к унаследованному методу, вызывая
при этом реализацию суперкласса

```
45 class Animal {
46     func makeSound() {
47         print("Animal makes a sound")
48     }
49 }
50
51 class Dog: Animal {
52     override func makeSound() {
53         print("Woof!")
54     }
55 }
56
57 class Cat: Animal {
58     override func makeSound() {
59         print("Meow!")
60     }
61 }
62
63 let animal = Animal()
64 animal.makeSound()
65
66 let dog = Dog()
67 dog.makeSound()
68
69 let cat = Cat()
70 cat.makeSound()
```



Animal makes a sound
Woof!
Meow!

☐ "Animal makes a sound\n"

☐ "Woof!\n"

☐ "Meow!\n"

☐ Animal

☐ Animal

☐ Dog

☐ Dog

☐ Cat

☐ Cat

Наследование: Использование super

Когда super полезен:



Расширение поведения

Подкласс хочет добавить свое поведение к методу, но при этом сохранить и выполнить оригинальное поведение суперкласса.



Инициализация

При инициализации подкласса часто требуется вызвать инициализатор суперкласса для правильной настройки унаследованных свойств.

```
73
74 class TransportVehicle {
75     func move() {
76         print("Transport vehicle starts
              moving")
77     }
78 }
79
80 class Car: TransportVehicle {
81     override func move() {
82         super.move()
83         print("Car is driving fast!")
84     }
85 }
86
87 let car = Car()
88 car.move()
```

☐ "Transport vehicle starts moving\n"

☐ "Car is driving fast!\n"

☐ Car

☐ Car



Transport vehicle starts moving
Car is driving fast!

Наследование: Ограничения и недостатки



Жесткая иерархия

Наследование создает жесткую иерархию классов, которая может быть трудноизменяемой и негибкой.



Проблема "хрупкого базового класса"

Изменение базового класса может сломать код в производных классах



Ограничение множественного наследования

Swift не поддерживает множественное наследование классов



Связанность (Coupling)

Наследование создает тесную связь между суперклассом и подклассом.

Введение в Протоколы

Протоколы: Основы и Назначение



Протоколы – это мощная концепция в Swift, лежащая в основе протокол-ориентированного программирования (POP). Протокол определяет контракт или набор требований, которым должен соответствовать тип.

Основные характеристики протоколов:



Определение требований

Протокол объявляет методы, свойства, инициализаторы и другие требования, которые conforming type (тип, соответствующий протоколу) должен реализовать.



Независимость от типа

Протокол определяет что нужно делать, а не как это делать. Разные типы могут по своему реализовывать один протокол



Множественное соответствие

Тип может соответствовать нескольким протоколам, комбинируя различные наборы требований.



Типы первого класса

Протоколы в Swift являются типами первого класса, их можно использовать как типы данных, передавать в функции, возвращать из функций и т.д.

Протоколы: Объявление протокола



Протоколы объявляются с использованием ключевого слова `protocol`, за которым следует имя протокола. Внутри фигурных скобок `{}` определяются требования протокола.

```
91
92 protocol ProtocolName {
93     // Protocol requirements:
94     // - Properties
95     // - Methods
96     // - Initializers
97     // - etc.
98 }
99
100 protocol Flying {
101     var flightAltitude: Double { get set }
102     func takeOff()
103     func land()
104 }
105
106 protocol Movable {
107     var speed: Double { get set }
108     func move()
109     func stop()
110 }
```

Протоколы: Методы и Свойства



Протоколы могут требовать от соответствующих типов реализацию определенных методов и свойств.

```
138 protocol Describable {
139     var name: String { get }
140     func description() -> String
141 }
142
143 struct Product: Describable {
144     let name: String
145     let price: Double
146
147     func description() -> String {
148         return "Product: \(name), Price: \(price)"
149     }
150 }
151
152 class Event: Describable {
153     var name: String
154     var date: Date
155
156     init(name: String, date: Date) {
157         self.name = name
158         self.date = date
159     }
160
161     func description() -> String {
162         return "Event: \(name), Date: \(date)"
163     }
164 }
165
166 let product = Product(name: "Apple", price: 1.0)
167 print(product.description())
168
169 let event = Event(name: "Birthday", date: Date())
170 print(event.description())
```

☐ "Product: Apple, Price: 1.0"

☐ "Event: Birthday, Date: 2025-03-10 15:05:18 +0000"

☐ Product

☐ "Product: Apple, Price: 1.0\n"

☐ Event

☐ "Event: Birthday, Date: 2025-03-10 15:05:18 +0000\n"

Product: Apple, Price: 1.0

Event: Birthday, Date: 2025-03-10 15:05:18 +0000

Протоколы: Инициализаторы

Важные моменты:



1

Если класс соответствует протоколу, требующему инициализатор, он должен реализовать этот инициализатор как `required initializer` (`required init`).



2

Если структура или перечисление соответствуют протоколу с инициализатором, `required` не требуется, так как структуры и перечисления не могут быть наследованы.

```
174 protocol Nameable {
175     init(name: String)
176 }
177
178 class Creature: Nameable {
179     let name: String
180
181     required init(name: String) {
182         self.name = name
183     }
184 }
185
186 struct Planet: Nameable {
187     let name: String
188
189     init(name: String) {
190         self.name = name
191     }
192 }
193
194 let creature = Creature(name: "Dragon")
195 print(creature.name)
196
197 let planet = Planet(name: "Mars")
198 print(planet.name)
```

☐ Creature
☐ "Dragon\n"

☐ Planet
☐ "Mars\n"

Dragon
Mars

Протоколы как типы данных

Использование протоколов как типов:

- ✓ Переменные и константы
- ✓ Параметры функций
- ✓ Возвращаемые значения функций
- ✓ Коллекции

```
201
202 protocol Printable {
203     func printDescription()
204 }
205
206 struct Book: Printable {
207     let title: String
208     func printDescription() {
209         print("Book: \(title)")
210     }
211 }
212
213 struct Magazine: Printable {
214     let title: String
215     func printDescription() {
216         print("Magazine: \(title)")
217     }
218 }
219
220 func printElement(element: Printable) {
221     element.printDescription()
222 }
223
224 let book = Book(title: "Master and Margarita")
225 let magazine = Magazine(title: "National Geographic")
226
227 printElement(element: book)
228 printElement(element: magazine)
229
230 let printableElements: [Printable] = [book, magazine]
231 for element in printableElements {
232     element.printDescription()
233 }
```

 "Book: Master and Margarita\n"

 "Magazine: National Geographic\n"

 Book

 Magazine

 Array of 2 Printable elements



```
Book: Master and Margarita
Magazine: National Geographic
Book: Master and Margarita
Magazine: National Geographic
```

Протоколы: Делегирование



Делегирование – это паттерн проектирования, в котором объект (делегат) отвечает на события или выполняет действия от имени другого объекта (делегирующего объекта). Протоколы играют ключевую роль в реализации делегирования в Swift.

```
237 protocol DataDownloaderDelegate: AnyObject {
238     func dataDownloaded(data: Data)
239     func dataDownloadFailedWithError(error: Error)
240 }
241
242 class DataDownloader {
243     weak var delegate: DataDownloaderDelegate?
244     func downloadData(url: URL) {
245         DispatchQueue.global().asyncAfter(deadline: .now() + 2) {
246             let success = Bool.random()
247             if success {
248                 let dummyData = "Data downloaded successfully!".data(using:
249                     .utf8)!
250                 self.delegate?.dataDownloaded(data: dummyData)
251             } else {
252                 let error = NSError(domain: "Download", code: 1, userInfo:
253                     [NSLocalizedStringKey: "Data download error"])
254                 self.delegate?.dataDownloadFailedWithError(error: error)
255             }
256         }
257     }
258 }
259
260 class DataController: DataDownloaderDelegate {
261     let downloader: DataDownloader
262
263     init(downloader: DataDownloader) {
264         self.downloader = downloader
265         self.downloader.delegate = self
266     }
267
268     func startDownload() {
269         let url = URL(string: "https://example.com/data")!
270         downloader.downloadData(url: url)
271     }
272
273     func dataDownloaded(data: Data) {
274         let text = String(data: data, encoding: .utf8)!
275         print("Data downloaded: \(text)")
276     }
277
278     func dataDownloadFailedWithError(error: Error) {
279         print("Download error: \(error.localizedDescription)")
280     }
281 }
282
283 let downloader = DataDownloader()
284 let controller = DataController(downloader: downloader)
285 controller.startDownload()
```

☐ true

☐ 29 bytes

☐ ()

☐ https://example.com/data

☐ DataDownloader

☐ "Data downloaded successfully!"

☐ "Data downloaded: Data downloaded successfully!\n"

☐ DataDownloader

☐ DataController

☐ DataController

Data downloaded: Data downloaded successfully!

ООП против ROP: Ключевые различия



В Swift часто используется комбинация ООП и ROP, используя наследование там, где оно уместно, и протоколы для обеспечения гибкости и композиции.

Характеристика	ООП (Объектно-ориентированное программирование)	ROP (Протокол-ориентированное программирование)
Основной принцип	Иерархия классов, наследование	Протоколы, композиция
Организация кода	Классы и объекты	Протоколы и типы, которые им соответствуют
Повторное использование кода	Наследование, полиморфизм	Композиция протоколов, расширения
Гибкость	Менее гибкое, жесткая иерархия	Более гибкое, за счет композиции
Связанность	Высокая связанность между классами	Низкая связанность, зависимость от протоколов
Типы данных	Классы (reference types)	Протоколы работают и с value types, и с reference types
Тестируемость	Может быть сложнее тестировать	Легче тестировать, возможность использовать мок-объекты на основе протоколов
Проблема "ромба"	Возможна при множественном наследовании	Отсутствует благодаря композиции

SOLID

Принципы SOLID



Принципы SOLID – это набор из пяти фундаментальных принципов объектно-ориентированного дизайна, разработанных Робертом Мартином и способствующих созданию поддерживаемого, гибкого и устойчивого к изменениям программного обеспечения.

Пять принципов SOLID:



S

Single Responsibility Principle (Принцип единственной ответственности)



O

Open/Closed Principle (Принцип открытости/закрытости)



L

Liskov Substitution Principle (Принцип подстановки Лисков)



I

Interface Segregation Principle (Принцип разделения интерфейса)



D

Dependency Inversion Principle (Принцип инверсии зависимостей)

SOLID - SRP (Single Responsibility Principle)

В контексте Swift и POP:

- ✓ Протоколы должны быть сфокусированы
- ✓ Типы должны реализовывать одну ответственность
- ✓ Разделение обязанностей

```
283 // Пример кода (нарушение SRP)
284 protocol DocumentManager {
285     func saveDocument(text: String)
286     func displayDocument()
287 }
288
289 // Пример кода (соблюдение SRP – разделение на два протокола):
290 protocol DocumentStorage {
291     func saveDocument(text: String)
292 }
293
294 protocol DocumentDisplay {
295     func displayDocument()
296 }
297
298 class DocumentManagerImplementation: DocumentStorage, DocumentDisplay {
299     func saveDocument(text: String) {
300         print("Document saved: \(text)")
301     }
302
303     func displayDocument() {
304         print("Document displayed")
305     }
306 }
```

SOLID - OCP (Open/Closed Principle)

В контексте Swift и POP:

- ✓ Расширение через протоколы
- ✓ Использование расширений (Extensions)
- ✓ Композиция вместо изменения

```
307 //Пример кода (нарушение OCP – модификация класса для добавления новой функциональности):
310 class Shape {
311     enum ShapeType {
312         case circle, square, triangle
313     }
314     let type: ShapeType
315
316     init(type: ShapeType) {
317         self.type = type
318     }
319
320     func draw() {
321         switch type {
322             case .circle:
323                 print("Drawing a circle")
324             case .square:
325                 print("Drawing a square")
326             case .triangle:
327                 print("Drawing a triangle")
328         }
329     }
330 }
331
332 //Пример кода (соблюдение OCP – расширение через протоколы и новые типы):
333 protocol DrawableShape {
334     func draw()
335 }
336
337 struct Circle: DrawableShape {
338     func draw() {
339         print("Drawing a circle")
340     }
341 }
342 struct Square: DrawableShape {
343     func draw() {
344         print("Drawing a square")
345     }
346 }
347 struct Triangle: DrawableShape {
348     func draw() {
349         print("Drawing a triangle")
350     }
351 }
352 struct Rectangle: DrawableShape {
353     func draw() {
354         print("Drawing a rectangle")
355     }
356 }
```

SOLID - LSP (Liskov Substitution Principle)

В контексте Swift и POP:

- ✓ Соответствие контракту протокола
- ✓ Предсказуемое поведение
- ✓ Избегание исключений или неверных результатов

```
359
360 protocol IntegerCollection {
361     func add(number: Int)
362     func getSum() -> Int
363 }
364
365 // Нарушение LSP: вызывает fatalError, что делает поведение неожиданным
366 class PositiveIntegerCollection: IntegerCollection {
367     private var numbers: [Int] = []
368
369     func add(number: Int) {
370         guard number >= 0 else { fatalError("Only positive numbers allowed") }
371         numbers.append(number)
372     }
373
374     func getSum() -> Int {
375         return numbers.reduce(0, +)
376     }
377 }
378
379 // корректное поведение, соблюдающее LSP
380 class SafeIntegerCollection: IntegerCollection {
381     private var numbers: [Int] = []
382
383     func add(number: Int) {
384         // Просто игнорируем отрицательные числа
385         if number >= 0 { numbers.append(number) }
386     }
387
388     func getSum() -> Int {
389         return numbers.reduce(0, +)
390     }
391 }
392
393 func calculateCollectionSum(collection: IntegerCollection) -> Int {
394     collection.add(number: 5)
395     collection.add(number: 10)
396     return collection.getSum()
397 }
398
399 let positiveCollection = PositiveIntegerCollection()
400 // calculateCollectionSum(collection: positiveCollection) // Может вызвать краш
401
402 let safeCollection = SafeIntegerCollection()
403 calculateCollectionSum(collection: safeCollection) // Работает корректно
```

Array of 2 Int elements

15

15

PositiveIntegerCollection

SafeIntegerCollection

15

SOLID - ISP (Interface Segregation Principle)

В контексте Swift и POP:

- ✓ Маленькие, сфокусированные протоколы
- ✓ Композиция протоколов
- ✓ Избегайте "протоколов общего назначения"

```
407 //Пример кода (нарушение ISP – "толстый" протокол):
408 protocol Device {
409     func print()
410     func scan()
411     func sendFax()
412 }
413
414 class MultifunctionPrinter: Device {
415     func print() { /* ... */ }
416     func scan() { /* ... */ }
417     func sendFax() { /* ... */ }
418 }
419
420 class Printer: Device {
421     func print() { /* ... */ }
422     func scan() {
423         // Empty implementation or exception
424     }
425     func sendFax() {
426         // Empty implementation or exception
427     }
428 }
429
430 //Пример кода (соблюдение ISP – разделение на протоколы по функциям):
431 protocol PrintingDevice {
432     func print()
433 }
434
435 protocol ScanningDevice {
436     func scan()
437 }
438
439 protocol FaxDevice {
440     func sendFax()
441 }
442
443 class MultifunctionPrinter2: PrintingDevice, ScanningDevice, FaxDevice {
444     func print() { /* ... */ }
445     func scan() { /* ... */ }
446     func sendFax() { /* ... */ }
447 }
448
449 class Printer2: PrintingDevice {
450     func print() { /* ... */ }
451 }
```

SOLID - DIP

(Dependency Inversion Principle)

В контексте Swift и POP:

- ✓ Зависимость от протоколов, а не от конкретных типов
- ✓ Протоколы как абстракции
- ✓ Инъекция зависимостей

```
455 //Пример кода (нарушение DIP – прямая зависимость от конкретного типа):
456 class NewsService {
457     func fetchLatestNews() -> [String] {
458         return ["News 1", "News 2"]
459     }
460 }
461 class NewsNotifier {
462     let service = NewsService()
463     func notifyAboutNews() {
464         let news = service.fetchLatestNews()
465         print("New news: \(news)")
466     }
467 }
468
469 //Пример кода (соблюдение DIP – зависимость от протокола "NewsSource"):
470 protocol NewsSource {
471     func fetchLatestNews() -> [String]
472 }
473 class NewsAPIService: NewsSource {
474     func fetchLatestNews() -> [String] {
475         return ["News API 1", "News API 2"]
476     }
477 }
478 class LocalNewsService: NewsSource {
479     func fetchLatestNews() -> [String] {
480         return ["Local news 1", "Local news 2"]
481     }
482 }
483 class NewsNotifier2 {
484     let source: NewsSource
485
486     init(source: NewsSource) { self.source = source }
487     func notifyAboutNews() {
488         let news = source.fetchLatestNews()
489         print("New news: \(news)")
490     }
491 }
492 let apiSource = NewsAPIService()
493 let localSource = LocalNewsService()
494
495 let apiNotifier = NewsNotifier2(source: apiSource)
496 apiNotifier.notifyAboutNews()
497
498 let localNotifier = NewsNotifier2(source: localSource)
499 localNotifier.notifyAboutNews()
```

Array of 2 String elements

Array of 2 String elements

Array of 2 String elements
"New news: ["Local news 1", "Local news 2"]\n"

NewsAPIService
LocalNewsService

NewsNotifier2
NewsNotifier2

NewsNotifier2
NewsNotifier2



New news: ["News API 1", "News API 2"]
New news: ["Local news 1", "Local news 2"]

Extensions

Расширения: Расширение ВОЗМОЖНОСТЕЙ ТИПОВ



Расширения (Extensions) – это мощная возможность Swift, позволяющая добавлять новую функциональность к существующим типам данных, таким как классы, структуры, перечисления и протоколы.

Возможности расширений:



Добавление вычисляемых свойств

Можно добавлять вычисляемые свойства (computed properties) к существующим типам.



Добавление методов экземпляра и типа

Можно добавлять новые методы экземпляра (instance methods) и типа (type methods).



Добавление новых инициализаторов

Можно добавлять convenience initializers (удобные инициализаторы) для классов.



Соответствие протоколам

Можно делать существующие типы соответствующими протоколам.



Организация кода

Расширения можно использовать для организации кода, разделяя функциональность типа на логические блоки в разных расширениях.

Расширения: Добавление функциональности



Расширения позволяют добавлять новую функциональность к существующим типам, не изменяя их исходный код. Это особенно полезно для добавления методов и вычисляемых свойств.

```
502
503 extension Int {
504     var isEven: Bool {
505         return self % 2 == 0
506     }
507
508     func squared() -> Int {
509         return self * self
510     }
511
512     static func random(digits: Int) -> Int {
513         let min = Int(pow(10.0, Double(digits - 1)))
514         let max = Int(pow(10.0, Double(digits)) - 1)
515         return Int.random(in: min...max)
516     }
517 }
518
519 let number = 7
520
521 print("\(number) is even: \(number.isEven)")
522 print("\(number) squared: \(number.squared())")
523
524 let randomNumber = Int.random(digits: 3)
525 print("Random 3-digit number: \(randomNumber)")
```



```
7 is even: false
7 squared: 49
Random 3-digit number: 782
```

☐ false

☐ 49

☐ 100

☐ 999

☐ 782

☐ 7

☐ "7 is even: false\n"

☐ "7 squared: 49\n"

☐ 782

☐ "Random 3-digit number: 782\n"

Расширения: Соответствие протоколам



Расширения также позволяют делать
существующие типы
соответствующими протоколам.

```
529 protocol TextDescriptable {
530     var textRepresentation: String { get }
531 }
532
533 extension Int: TextDescriptable {
534     var textRepresentation: String {
535         return "Integer: \(self)"
536     }
537 }
538
539 extension String: TextDescriptable {
540     var textRepresentation: String {
541         return "Text: \"\((self)\""
542     }
543 }
544
545 let number = 42
546 let string = "Hello"
547
548 print(number.textRepresentation)
549 print(string.textRepresentation)
550
551 func printTextDescription(describable: TextDescriptable) {
552     print(describable.textRepresentation)
553 }
554
555 printTextDescription(describable: number)
556 printTextDescription(describable: string)
```

☐ "Integer: 42"

☐ "Text: "Hello""

☐ 42

☐ "Hello"

☐ "Integer: 42\n"

☐ "Text: "Hello"\n"

☐ "Text: "Hello"\n"

Integer: 42
Text: "Hello"
Integer: 42
Text: "Hello"

Расширения: Организация кода



Расширения могут быть использованы для организации кода, разделяя функциональность типа на логические блоки. Это делает код более читабельным и поддерживаемым, особенно для больших типов с множеством функций.

```
559
560 class LargeClass {
561     // Many properties and methods...
562
563     // Core functionality of the class
564     func method1() { /* ... */ }
565     func method2() { /* ... */ }
566     // ...
567 }
568
569 extension LargeClass { // Extension for network functionality
570     func performNetworkRequest() { /* ... */ }
571     func handleNetworkResponse() { /* ... */ }
572 }
573
574 extension LargeClass { // Extension for UI functionality
575     func updateUI() { /* ... */ }
576     func showUserMessage() { /* ... */ }
577 }
```

Классовые протоколы: Ограничение соответствия классами



В Swift существуют классовые протоколы – протоколы, которые могут быть приняты только классами, а не структурами или перечислениями.

Когда использовать:



Когда требуется reference semantics



Для делегирования с weak ссылками

```
580
581 // @objc-протокол: поддерживает Objective-C runtime (selectors, weak references)
582 @objc protocol MyClassProtocol {
583     func doSomething()
584 }
585
586 class MyClass: MyClassProtocol {
587     func doSomething() {
588         print("MyClassProtocol method called")
589     }
590 }
591
592 // Классовый протокол без @objc (нативный Swift-подход)
593 protocol AnotherClassProtocol: AnyObject {
594     func performAction()
595 }
596
597 class AnotherClass: AnotherClassProtocol {
598     func performAction() {
599         print("AnotherClassProtocol method called")
600     }
601 }
602
603 let obj1: MyClassProtocol = MyClass()
604 obj1.doSomething()
605
606 let obj2: AnotherClassProtocol = AnotherClass()
607 obj2.performAction()
```



```
MyClassProtocol method called
AnotherClassProtocol method called
```

Классовые протоколы: Пример кода

```
611 @objc protocol EventHandler: AnyObject {
612     @objc optional func eventOccurred(data: String)
613 }
614
615 class EventGenerator {
616     weak var handler: EventHandler?
617
618     func generateEvent() {
619         let eventData = "Important event occurred!"
620         handler?.eventOccurred?(data: eventData)
621     }
622 }
623
624 class EventObserver: EventHandler {
625     func eventOccurred(data: String) {
626         print("Event registered: \(data)")
627     }
628 }
629
630 let generator = EventGenerator()
631 let observer = EventObserver()
632
633 generator.handler = observer
634
635 generator.generateEvent()
```

☐ "Important event occurred!"
☐ ()

☐ "Event registered: Important event occurred!\n"

☐ EventGenerator
☐ EventObserver

☐ EventGenerator

☐ EventGenerator

Event registered: Important event occurred!

Композиция протоколов



Композиция протоколов – это мощная возможность Swift, позволяющая создавать новые, составные протоколы, комбинируя несколько существующих протоколов.

Преимущества композиции протоколов:



Создание сложных контрактов

Композиция позволяет создавать протоколы, описывающие сложные наборы требований, путем комбинирования более простых, специализированных протоколов.



Гибкость и повторное использование

Можно повторно использовать существующие протоколы для создания новых, более специализированных, без дублирования кода.



Более читаемый код

Разбиение сложных требований на несколько протоколов и их композиция может сделать код более читабельным и понятным.

```
638
639 //Синтаксис определения
640 protocol Protocol1 {}
641 protocol Protocol2 {}
642 protocol Protocol3 {}
643 protocol CompositeProtocol: Protocol1, Protocol2, Protocol3 {}
644 typealias CompositeProtocol1 = Protocol1 & Protocol2 & Protocol3
645 |
```

Композиция протоколов: Пример кода

```
647
648 protocol Flying {
649     var flightAltitude: Double { get set }
650     func takeOff()
651     func land()
652 }
653
654 protocol Movable {
655     var speed: Double { get set }
656     func move()
657     func stop()
658 }
659
660 typealias FlyingAndMovable = Flying & Movable
661
662 struct Airplane: FlyingAndMovable {
663     var flightAltitude: Double = 0.0
664     var speed: Double = 0.0
665
666     func takeOff() { print("Airplane takes off") }
667     func land() { print("Airplane lands") }
668     func move() { print("Airplane moves on runway") }
669     func stop() { print("Airplane stops") }
670 }
671
672 func checkCapabilities(entity: FlyingAndMovable) {
673     print("Entity can fly and move.")
674     entity.takeOff()
675     entity.move()
676 }
677
678 let airplane = Airplane()
679 checkCapabilities(entity: airplane)
```

- ☐ "Airplane takes off\n"
- ☐ "Airplane moves on runway\n"
- ☐ "Entity can fly and move.\n"
- ☐ Airplane

Entity can fly and move.
Airplane takes off
Airplane moves on runway

Any Type: Универсальный ТИП



Any – это специальный протокол в Swift, представляющий собой универсальный тип, который может представлять экземпляр любого типа, включая классы, структуры, перечисления и функции.

Характеристики Any:

- ✓ **Универсальность**
Any может хранить значения любого типа
- ✓ **Потеря информации о типе**
При использовании Any, информация о конкретном типе значения теряется.
- ✓ **Использование в крайних случаях**

Когда Any может быть полезен:

- ✓ **Работа с heterogeneous collections**
Any может хранить значения любого типа
- ✓ **Интероперабельность с Objective-C API**
Некоторые Objective-C API могут возвращать значения типа id, который импортируется в Swift как Any.
- ✓ **Внутренняя реализация некоторых API**
Внутри Swift, Any может использоваться для реализации универсальных API, например, для работы с метатипами.

Any Type:

Использование и риски

Риски использования Any:

- ✓ Потеря type safety
- ✓ Больше кода для type casting
- ✓ Снижение производительности

```
682
683 var data: [Any] = []
684
685 data.append(42)
686 data.append("String")
687 data.append(true)
688 data.append({ print("Function in Any array") })
689
690 for element in data {
691     if let number = element as? Int {
692         print("Integer: \(number * 2)")
693     } else if let string = element as? String {
694         print("String: \(string.uppercased())")
695     } else if let boolean = element as? Bool {
696         print("Boolean: \(boolean ? "True" : "False")")
697     } else if let function = element as? () -> Void {
698         function()
699     } else {
700         print("Unknown type")
701     }
702 }
```



```
Integer: 84
String: STRING
Boolean: True
Function in Any array
```

- ☐ Array of 0 Any elements
- ☐ Array of 1 Any element
- ☐ Array of 2 Any elements
- ☐ Array of 3 Any elements
- ☐ Array of 4 Any elements, ()

☐ "Integer: 84\n"

☐ "String: STRING\n"

☐ "Boolean: True\n"

AnyObject Type



AnyObject – это протокол в Swift, представляющий собой тип "любой классовый тип". В отличие от Any, AnyObject может представлять экземпляр только классowego типа (reference type).

Характеристики AnyObject:



Ограничение классовыми типами

AnyObject может хранить только экземпляры классов.



Reference semantics

Значения типа AnyObject всегда являются reference types.

Когда AnyObject может быть полезен:



Работа с Objective-C API:

При взаимодействии с Objective-C кодом



Коллекции объектов

Для создания коллекций, которые должны хранить только объекты классов (reference types).



Динамическая диспетчеризация

В некоторых случаях, AnyObject может использоваться для динамической диспетчеризации методов в runtime

AnyObject Type: Использование и ограничения

Ограничения AnyObject:

- ✓ Ограничение классовыми типами
- ✓ Type safety
- ✓ Reference semantics only

```
705
706 class ClassA {}
707 class ClassB {}
708 struct StructC {}
709
710 var objects: [AnyObject] = []
711
712 objects.append(ClassA())
713 objects.append(ClassB())
714 // objects.append(StructC()) // Compile error!
715
716 for object in objects {
717     if let classA = object as? ClassA {
718         print("Object of type ClassA")
719     } else if let classB = object as? ClassB {
720         print("Object of type ClassB")
721     } else {
722         print("Unknown class type")
723     }
724 }
```



Object of type ClassA
Object of type ClassB

- ☐ Array of 0 AnyObject elements
- ☐ Array of 1 AnyObject element
- ☐ Array of 2 AnyObject elements
- ☐ "Object of type ClassA\n"
- ☐ "Object of type ClassB\n"

Экзистенциальный контейнер



Экзистенциальный контейнер – это структура данных, которая "оборачивает" экземпляр типа, соответствующего протоколу.

Экзистенциальный контейнер:



Обертка для conforming type



Метаданные протокола

Контейнер хранит метаданные о протоколе (например, указатели на таблицу виртуальных методов – witness table) и о конкретном типе, который conform to protocol.



Динамическая диспетчеризация

При вызове метода протокола на переменной типа протокола, Swift использует информацию в экзистенциальном контейнере для динамической диспетчеризации вызова к фактической реализации метода в underlying type.

Экзистенциальный контейнер



Экзистенциальный контейнер – это структура данных, которая "оборачивает" экземпляр типа, соответствующего протоколу.

Как это работает при вызове метода протокола:

- ✓ Компилятор видит вызов метода на переменной типа протокола.
- ✓ Во время выполнения (runtime), Swift обращается к экзистенциальному контейнеру, связанному с переменной.
- ✓ Используя метаданные протокола и witness table, Swift определяет фактический тип underlying instance.
- ✓ Происходит динамическая диспетчеризация вызова к реальной реализации метода протокола для этого типа, найденной в witness table.

725			
726			
727	+-----+		
728	Метаданные протокола "МойПротокол"	-->	Описывают требования протокола (методы, свойства)
729	+-----+		
730	Указатель на witness table	-->	Таблица виртуальных методов (конкретная для типа)
731	+-----+		
732	Указатель на данные	-->	Ссылка на экземпляр типа, соответствующий протоколу
733	+-----+		

POP в стандартной библиотеке Swift



Стандартная библиотека Swift активно использует протокол-ориентированное программирование (POP) в своей архитектуре. Многие ключевые типы и функциональности построены на протоколах.

Примеры POP в стандартной библиотеке Swift:



Collection protocol family

Collection, Sequence, IteratorProtocol, Indexable, и др.



Equatable и Hashable protocols

Многие стандартные типы (Int, String, Array, Dictionary и т. д.) соответствуют этим протоколам.



Codable protocol

Протокол Codable является composition of Encodable и Decodable.



Comparable protocol

Стандартные числовые типы и строки соответствуют Comparable.



StringProtocol

Тип String и другие строковые типы (например, Substring) соответствуют StringProtocol.

Заключение: Сочетание POP и ООП



Swift позволяет использовать обе парадигмы – и ООП, и POP – и эффективно сочетать их в одном проекте.

Сочетание ООП и POP в Swift:



Наследование для иерархий типов

Наследование остается полезным инструментом для создания иерархий классов



Протоколы для гибкости и композиции

Используйте протоколы для определения контрактов, которые могут быть реализованы разными типами



Расширения для организации и расширения функциональности

Расширения позволяют добавлять новую функциональность к существующим типам



SOLID принципы для качественного дизайна:

Применяйте принципы SOLID как при использовании ООП, так и при использовании POP для создания поддерживаемого и гибкого кода.



Спасибо!



Вопросы?



Домашнее задание

